Music Library Query Engine

Technical Documentation

Author: AiPixelCode

Version: 1.0

June 2026

Introduction

Music Library Query Engine is a Python library designed to manage user and music album data while providing multiple query operations over the stored information.

The project separates data management from query operations, resulting in a simple, modular, and reusable architecture that can be integrated into larger applications.

Project Objectives

- Provide a simple interface for managing users and music albums.
- Separate data registration from query operations.
- Organize information using standard Python data structures.
- Build reusable and modular functions.

Features

The library provides the following functionality:

- Register new users.
- Register new music albums.
- Query purchased tracks by user and artist.
- Query purchased tracks by user and genre.
- Query purchased tracks by age and artist.
- Query purchased tracks by age and genre.
- Query purchased tracks by city and artist.
- Query purchased tracks by city and genre.

Data Model

User Structure

User information is stored using the following structure:

all_users = {username: {"age": int, "city": str, "albums": list[str]}}

Each user stores only the names of purchased albums. Album metadata is retrieved from the album collection.

Album Structure

Album information is stored independently:

all_albums = {album_name: {"artist": str, "genre": str, "tracks": int}}

Keeping album information separate eliminates duplicated data across multiple users.

API Reference

1. add_user()

Purpose: Registers a new user.

Parameters:

Name	Type
username	str
age	int
city	str
albums	list[str]
all_users	dict

Returns: None

2. add_album()

Purpose: Registers a new music album.

Parameters:

Name	Type
name	str
artist	str
genre	str
tracks	int
all_albums	dict

Returns: None

3. query_user_artist()

Purpose: Returns the total number of purchased tracks by a specific user from a given artist.

Returns: int

4. query_user_genre()

Purpose: Returns the total number of purchased tracks by a specific user within a given genre.

Returns: int

5. query_age_artist()

Purpose: Returns the total number of purchased tracks by users of a specified age from a given artist.

Returns: int

6. **query_age_genre()**

Purpose: Returns the total number of purchased tracks by users of a specified age within a given genre.

Returns: int

7. **query_city_artist()**

Purpose: Returns the total number of purchased tracks by users living in a specified city from a given artist.

Returns: int

8. **query_city_genre()**

Purpose: Returns the total number of purchased tracks by users living in a specified city within a given genre.

Returns: int

Design Decisions

The project maintains two independent collections: one for users and another for albums. Users store only references to purchased album names, while album metadata is maintained separately. This design minimizes data duplication and simplifies future updates.

Data registration and query operations are implemented as separate functions, following the principle of single responsibility. This improves readability, maintainability, and extensibility.

Query Results

The library provides integer values as the result of every query function.

Depending on the requested query, the returned value represents the total number of purchased tracks that satisfy the specified filtering criteria.

Typical outputs include:

- Total tracks purchased by a specific user from an artist
- Total tracks purchased by a specific user within a genre
- Total tracks purchased by users of a given age from an artist
- Total tracks purchased by users of a given age within a genre
- Total tracks purchased by users living in a city from an artist
- Total tracks purchased by users living in a city within a genre

All query functions return an integer value (int).

Future Improvements

- JSON-based persistence
- Database integration
- Update and delete operations
- Object-oriented implementation
- Query optimization for large datasets

Conclusion

Music Library Query Engine demonstrates the implementation of a modular Python library for managing structured data and performing multiple query operations. The project emphasizes simplicity, clear separation of responsibilities, and code reusability, providing a solid foundation for future extensions.